

Tipologie di esercizi

- → Induzione strutturale
- → Scoping statico e dinamico
 - descrizione della catena statica/CRT
 - completamento del codice
- Scoping + binding
- Passaggio di parametri
 - valore
 - riferimento
- Ricorsione e ricorsione in coda
- → Semantica di un'istruzione

Induzione

Alberi binari: BTree

$\begin{cases} \text{foglia} \in \text{BTree} \\ T_1, T_2 \in \text{BTree} \Rightarrow \text{branch}(T_1, T_2) \in \text{BTree} \end{cases}$

$l: \text{BTree} \rightarrow \mathbb{N}$ n° foglie

$h: \text{BTree} \rightarrow \mathbb{N}$ altezza

$$\begin{cases} l(\text{foglia}) = 1 \end{cases}$$

$$\begin{cases} l(\text{branch}(T_1, T_2)) = l(T_1) + l(T_2) \end{cases}$$

$$\begin{cases} h(\text{foglia}) = 0 \end{cases}$$

$$\begin{cases} h(\text{branch}(T_1, T_2)) = \max(h(T_1), h(T_2)) + 1 \end{cases}$$

Dimostrare che $\forall T \in \text{BTree} . l(T) \leq 2^{h(T)}$ ^{tesi}

Base: $T = \text{foglia}$

$$l(T) = 1 \quad h(T) = 0$$

$$l(T) = 1 = 2^0 = 2^{h(T)} \Rightarrow l(T) \leq 2^{h(T)}$$

Passo ind.: Hp. ind. $T_1, T_2 \in \text{BTree}$ t.c. $l(T_1) \leq 2^{h(T_1)}$ e $l(T_2) \leq 2^{h(T_2)}$

Dobbiamo dimostrare che se $T = \text{branch}(T_1, T_2)$ allora $l(T) \leq 2^{h(T)}$

$$l(T) = l(\text{branch}(T_1, T_2)) \stackrel{\text{def } l}{=} l(T_1) + l(T_2)$$

$$\stackrel{\text{Hp. ind.}}{\leq} 2^{h(T_1)} + 2^{h(T_2)} \quad *$$

$$h(T) = h(\text{branch}(T_1, T_2)) = \max(h(T_1), h(T_2)) + 1$$

prendiamo $h = \max(h(T_1), h(T_2)) \Rightarrow \begin{matrix} h \geq h(T_1) \\ h \geq h(T_2) \end{matrix}$

$$* \leq 2^h + 2^h = 2(2^h) = 2^{h+1} = 2^{\max(h(T_1), h(T_2)) + 1} = 2^{h(T)} \quad \checkmark$$

Scoping statico e dinamico : completamento codice

```
{ int a = 0;
```

⊛

```
while (a ≤ 1) {
```

```
    int x;
```

→ locale al while

⊛⊛

```
    x = fun();
```

```
    a++;
```

```
}
```

```
}
```

⇒ Per essere eseguibile dobbiamo definire fun() in ⊛

⇒ fun() deve avere un ambiente non locale perché le regole di scoping abbiano effetto sul risultato

⇒ Il riferimento non locale in fun() deve essere definito sia nel contesto di definizione di fun() che nel contesto di esecuzione di fun()

⊛ → $\begin{cases} \text{def. di } x \text{ + inizializzazione} \\ \text{def. di fun()} \end{cases} \begin{cases} \text{int } x = 2; \\ \text{int fun() } \{ \text{return } x; \} \end{cases}$ → per essere eseguibile dobbiamo inizializzare x

⊛⊛ → inizializzazione di x { x = 2;

[0:37:00]

```

{ int a=0;
  int x=2;
  int fun() { return x; }
  while (a<=1) {
    int x;
    x=a;
    x=fun();
    a++;
  }
}

```

Scoping statico

la chiamata di `fun()` restituisce la `x` globale che non viene mai modificata quindi in entrambe le chiamate `x` restituisce valore 2.

Scoping dinamico

la chiamata di `fun()` restituisce la `x` interna al `while` che alla prima iterazione vale `a=0` (globale non modificata prima della chiamata).

`a++` incrementa `a` a 1 la var. globale `a`
 $\Rightarrow$ nella seconda chiamata `fun()` restituisce la `x` interna al `while` che ora vale 1

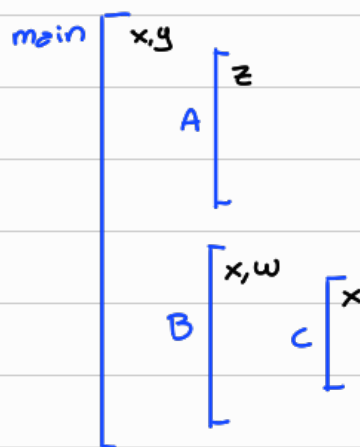
[0:44:00]

Scoping statico e dinamico : catena statica e CRT

```

{ int x=1; int y=2;
  void A() { int z=x+y; }
  void B() { int x=2; int w=4;
    void C() { int x=y+w;
      A(); }
    y=x+y;
    C(); }
  B(); }

```

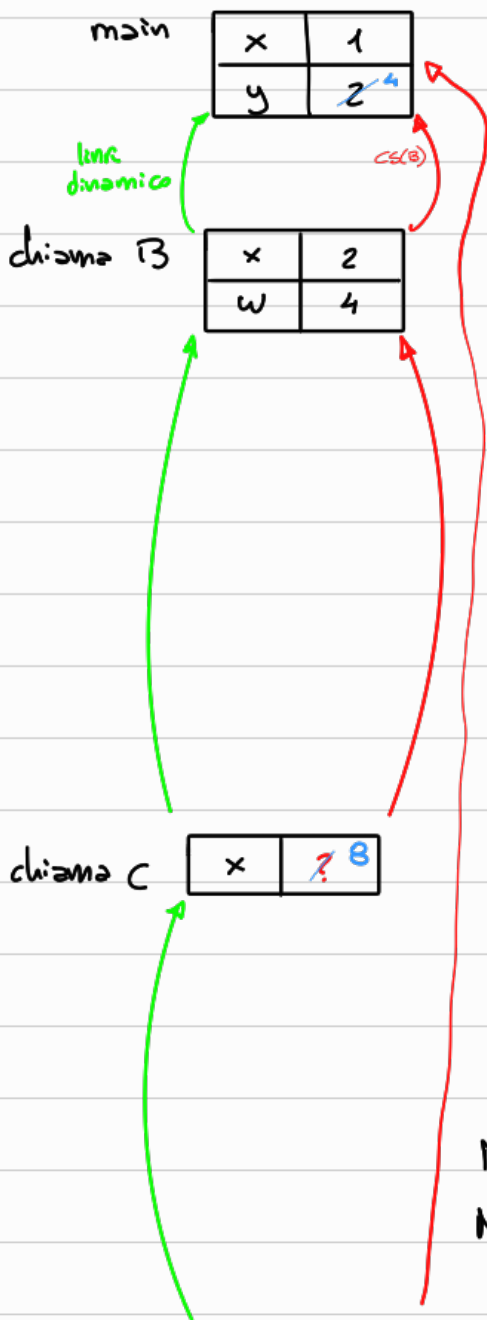


profondità dichiarazioni

$$\left\{ \begin{array}{l} Sd(main) = 0 \\ Sd(A) = Sd(B) = 1 \\ Sd(C) = 2 \end{array} \right.$$

seq. chiamate `main  $\rightarrow$  B  $\rightarrow$  C  $\rightarrow$  A`

Scoping statico



determiniamo il link statico

no di volte da risalire lungo la catena statica per trovare il genitore statico

$$K_B = Sd(chiamante) - Sd(chiamato) + 1$$

$$= Sd(main) - Sd(B) + 1 = 0 - 1 + 1 = 0$$

$$\Rightarrow CS(B) = main$$

esecuzione di B : $y = x + y$;

$$N_y = Sd(uso) - Sd(def)$$

blocco in cui usiamo y

il blocco che definisce y nella catena di annidamento

dice di quanto risalire la catena statica per trovare RDA del blocco che def y

$$N_y = Sd(B) - Sd(main) = 1 - 0 = 1 \Rightarrow y = 2$$

link statico

$$K_C = Sd(B) - Sd(C) + 1 = 1 - 2 + 1 = 0$$

$$\Rightarrow CS(C) = B$$

eseguimo C : $x = y + w \rightarrow 8$

$$N_y = Sd(C) - Sd(main) = 2 \Rightarrow y = 4$$

$$N_w = Sd(C) - Sd(B) = 1 \Rightarrow w = 4$$

link statico

$$K_A = Sd(C) - Sd(A) + 1 = 2 - 1 + 1 = 2$$

$$\Rightarrow CS(A) = main$$

esecuzione A : $z = x + y \rightarrow 5$

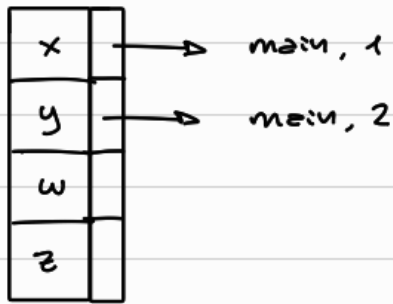
$$N_x = Sd(A) - Sd(main) = 1 - 0 = 1 \Rightarrow x = 1$$

$$N_y = Sd(A) - Sd(main) = 1 - 0 = 1 \Rightarrow y = 4$$

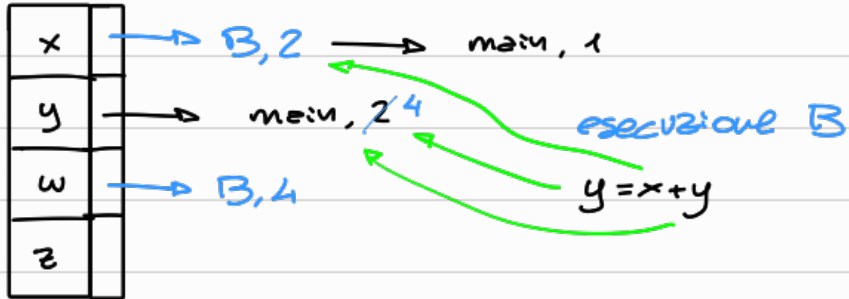
[4:40:00]

Dopo la chiamata tutte le chiamate terminano e lo stack deallocato

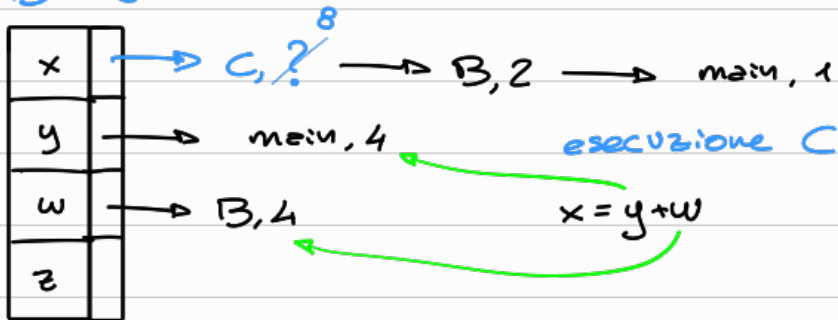
Scoping dinamico



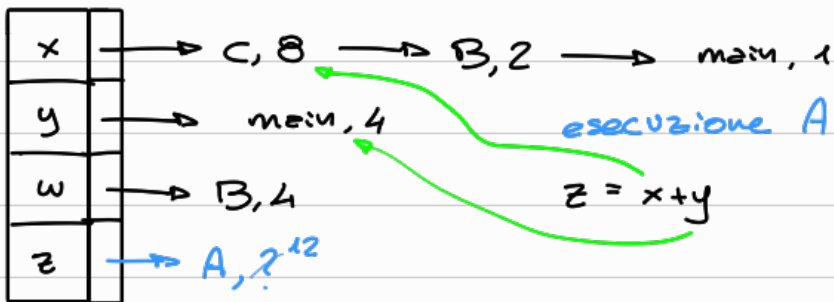
main → B



B → C



C → A



[1:20:00]

Altro esercizio nelle scan